

Instructions

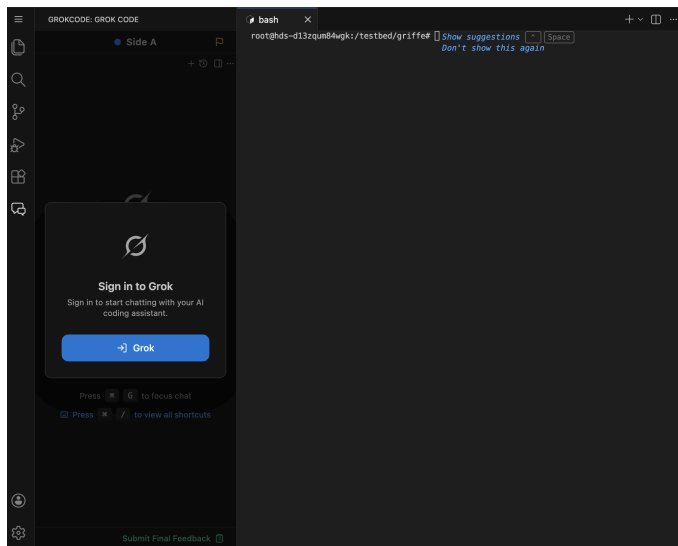
Project Proposals - HARD

What This Project Is

You create a hard coding prompt, test it with a preselected model in a Starfleet container, and submit the trace. Your prompts become future evaluation tasks. Goal: build a bank of genuinely difficult coding challenges that state-of-the-art models struggle with.

Step-by-Step Workflow

1. **Claim a task** from the [Project Proposals - HARD](#) project on Starfleet.
2. **Fill in your GitHub repo URL + commit hash.** Use a real, existing public repository. Use the latest commit on `main` unless you have a specific reason for an older one. The repo should remain public. Do not make task repos private. .
3. **Launch the container, wait for the Grok extension to show up** (on the left) and open up the terminal. **Do not sign in to Grok in the extension just yet.**



4. **Make sure that Grok uses the `sxs-claude-opus-4-6` model** by running the following command in the terminal shell:
`/root/.grok/bin/grok --model sxs-claude-opus-4-6`
Next, authenticate with the token, type `/exit` and only then proceed to Sign in to Grok through the extension.
5. **Sign in the Grok extension** (you must use the token, because Grok has no internet access).
6. **Install dependencies** needed to run the test suite or build the application before making any prompts.
7. **Send your prompt** to the model in the container conversation. The trace is captured automatically.

8. You may send “**continue**” if the model stalls, but do **not** send follow-up prompts that change the task. Evaluate based on the final result.
9. Click on “**Submit Final Feedback**” on the extension view to submit the feedback
10. **Fill out the submission form** (details below).
11. **Submit the task.**

In case of any doubts, check the [FAQ](#).

Do not run anything locally. All work is to be done within the Starfleet container.

Submission Form Fields

- **Repository Description** — Brief description of the repo. You can copy from the repo's GitHub "About" section.
- **Domain** — Select the closest match (Backend/API, Data Science/ML, Systems, DevOps, Security, Mobile, Other). Be careful to not do anything related to visuals (frontend for Webdev and visual-gameplay for Game Dev)
- **Prompt** — The exact prompt you sent to the model. Must be what you actually typed in the container.
- **Difficulty Explanation** — 2-4 sentences on why this is hard. Be specific: where do models typically fail? What makes the correct solution non-obvious?
- **Rubrics** — At least 3 specific criteria describing the ideal solution. These will be used by reviewers to evaluate answers. Write them yourself — be specific enough that another person could apply them.
- **Did the model solve it?** — Yes (fully solved) / Partially (some issues) / No (failed).
- **Issues** (if partial or no) — What were the main problems in the model's solution?
- **How long did the model take for your selected turn?** — Select the time range. This is not the time it takes for your overall interaction or task. This is the time it took the model to solve your HARD turn.

What Makes a Good Proposal

Required:

- Hard enough that Opus takes >10 minutes **OR**
- Hard enough that Opus fails to solve the problem
- First prompt sent to the model (each submission is a fresh start)
- Real existing repository with a pinned commit hash
- At least 3 specific, evaluable rubrics
- Prompt must be in the model conversation inside the container
- The number of coding files(excluding files like README, docs) should be ~20+. This ensures that repo is not a toy project or an easy one and the task will load the model with large context.

NOT allowed:

- Visual related prompts:
 - a. UI-focused tasks Web apps
 - b. Game-Dev that requires visuals (engine programming and core systems are fine)
- Simple CRUD apps
- Docker or external APIs (e.g., Stripe)
- 0-to-1 / fresh repos (not yet supported)
- LLM-generated prompts — write them naturally as you would when coding

Good prompts:

- Complex refactors across multiple files
- Performance optimization with specific constraints
- Difficult algorithm implementations
- Bug fixes requiring deep codebase understanding
- Systems-level work (networking, concurrency, memory management)

Key Rules

- **One prompt per submission.** Each submission is independent — no carrying over context from previous submissions.
- **You can reuse repos** — 2-3 prompts on the same repo is OK, but different repos are preferred for distribution.
- **You don't need to fix the model's failures.** If the model fails on your hard prompt, that's expected.
- **Keep working while waiting.** Submit multiple proposals. Don't wait for reviews. Reviewers are working through the backlog.
- **Rubrics are YOUR job.** Write specific expectations for the ideal solution. Don't ask Cosmo or the model to generate them.

Prompt & Turn Clarifications

Dependencies & Setup

Dependencies should be installed using the language preset + whatever the repo ships. You can run normal shell commands (apt install, pip install, npm install, etc.) to check and set up before you start prompting. Do **NOT** use a turn to have the model install dependencies.

Multi-Turn vs Single-Turn

Most tasks should be single turn: one hard prompt, one model attempt.

Multi-turn is allowed ONLY when earlier turns build necessary context for the hard prompt (e.g., asking the model to explain the codebase structure before giving the real task). The turns before the hard prompt must be set up/context.

The prompt you paste into the Starfleet form must be the hard prompt itself.

Analyzing the Codebase

You can analyze the codebase locally or in a separate Grok session before crafting your prompt. This is encouraged. It helps you write better, more targeted prompts.

The key constraint: the prompt you submit should work as a realistic first message to the model in the container. It should not depend on prior conversation context that isn't captured in the trace.

Relationship to Single Turn Rating

Proposals are about collecting ideas. Single Turn Rating is the evaluation of 3 models' performance on your idea. The first prompt should be useful and realistic, not a setup turn.

Rubrics

Rubrics should be self contained, true/false, conditions that we can apply to the changes made by the model. The rubrics should be clear, objective and relevant to the prompt. Use examples if anything might be unclear. Examples:

Example Coding Prompt:

Fix the memory leak in `process_large_file(filename)` by ensuring the file is always closed, even on exceptions. Use a context manager. Do not change the processing logic.

Example Rubric:

The file should be closed. The changed code uses `a with open(...) as f:` block (or equivalent `contextlib` context manager) around all file operations, and there is no bare `open()` call without a matching `.close()` in a `finally` block in the modified function.

Prompt examples

None

We've been dealing with a slow memory leak in our Temporal TypeScript workers for the past couple of weeks. The growth is gradual(just a few MB per day per worker) ,but it's consistent,

and restarts clear it. The tricky part is that we can't reproduce it locally; it only shows up in production.

We've already ruled out a few obvious suspects. The activities themselves seem fine and workers under steady load stay flat. Workflow caching is tuned and bounded. We even ran our activity implementations outside Temporal for a week and saw no growth, so it doesn't look like our application code.

Heap snapshots suggest something is being retained inside the SDK, but it's hard for us to interpret. It looks like some objects are sticking around longer than they should, possibly across task boundaries. We're not sure if this is something we're doing wrong with the SDK or an actual bug.

Could you help us understand the root cause and the minimal fix?

None

I have a Socket.io app and I need each connected client to have a rate limit per event type where premium clients get higher limits and their events are processed before regular ones under heavy load. The rate limiting and priority system must be implemented at the lowest possible level inside the engine not at the application layer, clients must be authenticated and their tier cryptographically signed so no client can claim a higher tier than issued, when a client exceeds their rate limit their packets should be queued not dropped and dequeued in priority order as capacity frees up, this is important starvation must be prevented so lower tier clients are guaranteed processing within a configurable time window even under a sustained flood of premium traffic, please proceed with the initial plan and the coding changes also I need real time metrics per client per event type showing current rate, queue depth and wait time, do not break the existing socket.io functionality.

Checklist

Tutor Checklist

- ☐ The prompt is realistic, this is a natural prompt that a real developer would ask
 - ☐ The prompt does not add arbitrary constraints / additions
 - ☐ All constraints / additions are natural relative to the prompt
 - ☐ The prompt does not contain an endless list of constraints, details or specify how to do every little thing
 - ☐ The prompt requests something of value (no toy or inconsequential asks)
 - ☐ The prompt does not request something that would not be a realistic addition to the repository, e.g. adding a custom data pipeline directly onto the pandas repository
- ☐ The prompt should contain minimal details on how to implement it.
- ☐ The prompt has a concrete goal
 - ☐ The prompt's final goal is unambiguous
 - ☐ The prompt does not ask the model to find something wrong with the codebase
- ☐ The repo is sufficiently challenging
 - ☐ The repo is not empty or a toy project
 - ☐ The repo has meaningful complexity, complex tasks can be derived from it
 - ☐ Opus either can't solve the problem, or takes longer than 10min to solve it
- ☐ The rubrics make it clear what an ideal solution looks like
 - ☐ The rubrics clearly define what an ideal response looks like and are free of ambiguities such as requests for (clean modular code, detailed response)
 - ☐ They will be used in the future as a reward rubric during RL and will be used to create verifier methods for reward function and possibly also fed to GraderAgent as an eval.
 - ☐ So all rubrics that we write should be something that a verifier function or an agent could verify as well.
 - ☐ Let's take a rubric: `The feature XYZ is properly implemented`. This is bad because the word `proper` is subjective and will vary model to model. So rubrics shouldn't have subjective assertions

Bad:

- ☐ `There should be enough test cases for the feature XYZ -- again subjective`

Good:

- ☐ `The tests should check the edge cases EC1, EC2 & EC3 -- this is good.`
- ☐ `On a refactoring task: The methods exported/exposed by the package/class shouldn't change.`

- ☐ The prompt was not generated by an LLM
 - ☐ The prompt is written in natural tone and is not overly explicit in asks, this prompt resembles my own personal prompts to LLM models

- ☐ The agent does not require network dependencies/docker/containerisation/local LLMs to complete the task
- ☐ The prompt does not work with basic CRUD/Demo applications
- ☐ I am fully confident in my ability to assess the results of the prompt
- ☐ I did not use an open source issue that already has a merged solution - since the merged solution is most likely to be scrapped already it's more likely for the model to solve it easily.

FAQ

FAQ

1. What should I do when I see ``GROK_TUTOR_MODEL_ID is not configured. Please report this issue.`

Click to watch a [video on how to fix](#)

Steps:

- Unclaim the current task and start a new one
- Open the **container/sandbox**, and inside the terminal run
`/root/.grok/bin/grok --model sxs-claude-opus-4-6`
- Wait until the model on the bottom right is shown as `sxs-claude-opus-4-6` (you can type `a` to update the TUI) (see the image)
- Authenticate to the extension and use it to complete the task

 **DON'T** use the Grok Build CLI to submit prompts. Use only the **EXTENSION** to submit your prompts 

2. What should I do when **Proposals are stuck on "pending"**
Reviewers are working through a backlog. Keep submitting new ones.
3. What should I do when **Model subagents fail / errors during trace**
This counts as a valid attempt. Note the failure in the issues field.
4. I get **"Invalid request body: missing max_tokens"**
Known CLI issue. Try the `sxs opus 4.6` model instead.
5. **The model takes very long**
Work on another submission while waiting. You can have multiple sessions.
6. What should I do when the **Submit button is disabled**
Make sure you've hit the `Submit Final Feedback` in the VS Code session. This is a requirement before you can submit in Starfleet.
7. Why do I not see my proposals in <https://autoqa.teachx.ai/ttl/proposals/>
You only see the projects submitted through google forms here, For the one submitted via Startfleet is accessible only on starfleet.